Causal Transparency Algebra $\mathbb{R}_\varnothing$: definitions, structure and limits

Hakim Guettal

Technical note for `rnull-framework` v1.1.0
November 2026

Abstract

$\mathbb{R}_\varnothing = \mathbb{R} \sqcup \{\varnothing\}$ extends the real numbers by one element $\varnothing$, read as "no value was supplied", which is declared to be the identity of both an extended addition $\oplus$ and an extended multiplication $\otimes$. This note states precisely what structure results, proves the properties the software relies on, and separates them from claims the construction does not support. In short: $(\mathbb{R}_\varnothing, \oplus)$ and $(\mathbb{R}_\varnothing, \otimes)$ are commutative monoids obtained by adjoining an identity; together they are not a semiring, and the failure of distributivity is characterised exactly. With the absorbing product $\odot$ instead, $(\mathbb{R}_\varnothing, \oplus, \odot)$ *is* a commutative semiring, and a linear layer that skips missing inputs is the matrix product over it. Numerically, that layer coincides with zero imputation plus an "everything was missing" flag. Measured results are reported in Section 7; they show no accuracy advantage over imputation on the problems tested.

1 Definitions

Definition 1. Let $\varnothing \notin \mathbb{R}$ and $\mathbb{R}_\varnothing = \mathbb{R} \sqcup \{\varnothing\}$. For $a, b \in \mathbb{R}_\varnothing$,

$$a \oplus b = \begin{cases} a + b & a, b \in \mathbb{R} \\ a & b = \varnothing \\ b & a = \varnothing \end{cases} \qquad a \otimes b = \begin{cases} a\,b & a, b \in \mathbb{R} \\ a & b = \varnothing \\ b & a = \varnothing \end{cases}$$

(so $\varnothing \oplus \varnothing = \varnothing \otimes \varnothing = \varnothing$). Negation is $-\varnothing = \varnothing$ and $a \ominus b = a \oplus (-b)$. Two further products are used for the coupling of a weight w with an input x:

$$w \odot x = \begin{cases} w\,x & w, x \in \mathbb{R} \\ \varnothing & \text{otherwise} \end{cases} \qquad w \rhd x = \begin{cases} w \otimes x & x \in \mathbb{R} \\ \varnothing & x = \varnothing \end{cases}$$

and division is $a \oslash b = a/b$ for $a, b \in \mathbb{R}$, $b \neq 0$, and $\varnothing$ if either operand is $\varnothing$. The *tag* of a is $\nu(a) = 1$ if $a = \varnothing$ and 0 otherwise.

Division of a real number by 0 is left undefined by the algebra; the software follows IEEE 754 there.

2 Structure

Proposition 2. *$(\mathbb{R}_\varnothing, \oplus)$ and $(\mathbb{R}_\varnothing, \otimes)$ are commutative monoids with the same identity $\varnothing$.*

Proof. Let $B = (\{0,1\}, \wedge)$. The maps $\varphi_+ : \mathbb{R}_\varnothing \to (\mathbb{R}, +) \times B$, $x \mapsto (x, 0)$, $\varnothing \mapsto (0, 1)$, and $\varphi_\times : \mathbb{R}_\varnothing \to (\mathbb{R}, \cdot) \times B$, $x \mapsto (x, 0)$, $\varnothing \mapsto (1, 1)$, are injective, and a case check shows $\varphi_+(a \oplus b) = \varphi_+(a)\varphi_+(b)$ and $\varphi_\times(a \otimes b) = \varphi_\times(a)\varphi_\times(b)$. Their images are closed and contain the identities $(0, 1)$ and $(1, 1)$, so $\oplus$ and $\otimes$ inherit associativity and commutativity from the product monoids. □

Remark 3. This is the standard construction that adjoins an identity to a semigroup (written S^1 in semigroup theory, the `Option`/`Maybe` monoid in programming). The embeddings $\varphi_+, \varphi_\times$ are exactly the (value, mask) representation used by the code; they need different payloads for $\varnothing$ (0 and 1), which is why no single embedding into a product *ring* exists.

Proposition 4 (What changes relative to $\mathbb{R}$)**.** *1. 0 is no longer the additive identity of the whole set, but $0 \oplus x = x$ for $x \in \mathbb{R}$ and $0 \oplus \varnothing = 0$.*

2. $a \oplus b = \varnothing$ only if $a = b = \varnothing$. Hence no real number has an additive inverse, $(\mathbb{R}_\varnothing, \oplus)$ is not a group, and $x \ominus x = 0 \neq \varnothing$.

3. $0 \otimes a = 0$ for every $a \in \mathbb{R}_\varnothing$: zero still annihilates. In particular a measured zero in a product is not "skipped".

4. The only invertible element of $(\mathbb{R}_\varnothing, \otimes)$ is $\varnothing$.

Proposition 5 (Distributivity defect)**.** *For $a \in \mathbb{R}$ and $b, c \in \mathbb{R}_\varnothing$,*

$$\big((a \otimes b) \oplus (a \otimes c)\big) \ominus \big(a \otimes (b \oplus c)\big) = a \cdot \max\{\nu(b), \nu(c)\}.$$

Consequently $a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$ holds if and only if $a = \varnothing$, or $a = 0$, or $b, c \in \mathbb{R}$.

Proof. If $b, c \in \mathbb{R}$ this is distributivity in $\mathbb{R}$. If $b \in \mathbb{R}$, $c = \varnothing$: the left term is $ab + a$ and the right term is ab. If $b = c = \varnothing$: the left term is $a + a$ and the right term is $a \otimes \varnothing = a$. For $a = \varnothing$ both sides equal $b \oplus c$. □

Corollary 6. *$(\mathbb{R}_\varnothing, \oplus, \otimes)$ is not a semiring. The function $\chi(x) = \big((1 \otimes 0) \oplus (1 \otimes x)\big) \ominus \big(1 \otimes (0 \oplus x)\big)$ equals $\nu(x)$.*

Remark 7. χ is a consequence of the case distinctions already built into $\oplus$ and $\otimes$; it does not detect $\varnothing$ independently of them. In the (value, mask) representation it is the mask. In floating point the identity holds for finite x only: for $x = \pm\infty$ the expression is $\infty - \infty = \mathrm{NaN}$. The software therefore reads the mask and keeps the polynomial as a test.

Proposition 8 (Tag homomorphisms)**.** $\nu(a \oplus b) = \nu(a \otimes b) = \nu(a) \wedge \nu(b)$, $\nu(a \odot b) = \nu(a \oslash b) = \nu(a) \vee \nu(b)$, $\nu(w \rhd x) = \nu(x)$.

Proposition 9 (The semiring of the linear layer)**.** *$(\mathbb{R}_\varnothing, \oplus, \odot)$ is a commutative semiring with zero $\varnothing$ and unit 1, without zero divisors.*

Proof. $(\mathbb{R}_\varnothing, \odot)$ is the monoid $(\mathbb{R}, \cdot)$ with an absorbing element adjoined, so it is a commutative monoid with identity 1 and $\varnothing \odot a = \varnothing$. For distributivity, $w \odot (x \oplus y) = (w \odot x) \oplus (w \odot y)$: if $w = \varnothing$ both sides are $\varnothing$; if $w \in \mathbb{R}$ and exactly one of x, y is $\varnothing$, say y, both sides are wx; if both are $\varnothing$ both sides are $\varnothing$; otherwise it is distributivity in $\mathbb{R}$. Finally $a \odot b = \varnothing$ forces $a = \varnothing$ or $b = \varnothing$. □

Remark 10. This is the semiring obtained from $(\mathbb{R}, +, \cdot)$ by adjoining a new zero. Sums and matrix products over it are well defined and associative, so $y_i = \bigoplus_j W_{ij} \odot x_j$ is an ordinary matrix–vector product over $(\mathbb{R}_\varnothing, \oplus, \odot)$ and such layers compose as matrices do. The "bi-identity" product $\otimes$ plays no role in this layer; it is a separate fusion operator.

Proposition 11 (The directed coupling is a monoid action)**.** *For all $w, w', x, y \in \mathbb{R}_\varnothing$: $\varnothing \rhd x = x$, $(w \otimes w') \rhd x = w \rhd (w' \rhd x)$, $w \rhd (x \oplus y) = (w \rhd x) \oplus (w \rhd y)$, $w \rhd \varnothing = \varnothing$. That is, $\rhd$ is an action of the monoid $(\mathbb{R}_\varnothing, \otimes)$ on the monoid $(\mathbb{R}_\varnothing, \oplus)$ by endomorphisms.*

Remark 12. A null weight under $\rhd$ acts as the weight 1. This is a pass-through, not pruning: a pruned synapse must contribute nothing, which is what $\odot$ gives.

All statements of this section are checked exhaustively on a grid of representatives, and on random arrays, by `tests/test_algebra.py`.

3 Relation to existing practice

The (value, mask) pair with these rules is not new; what the project adds is a single vocabulary across software and a tagged hardware cell.

- **Reductions.** $\bigoplus$ and $\bigotimes$ over an array are the NaN-skipping reductions `nansum`/`nanprod` with the convention that an all-missing reduction is missing. This is also the behaviour of `numpy.ma` reductions and of SQL aggregates (`SUM` skips `NULL` and returns `NULL` on all-`NULL` input).
- **Element-wise operations.** $\odot$ and $\oslash$ propagate absence like IEEE NaN, SQL `NULL` and `numpy.ma` binary operations. $\oplus$ and $\otimes$ as *binary* operations are the less common choice.
- **Layers.** Mask-aware layers that renormalise by the number of valid inputs and propagate an updated mask are known from partial convolutions (Liu et al., ECCV 2018); missingness indicators as inputs are a standard baseline (e.g. GRU-D, Che et al., 2018).
- **Hardware.** Skipping multiply–accumulate operations for zero or absent operands is established in sparse accelerators (e.g. Cnvlutin, SCNN, Eyeriss).

The word "causal" in the project name is historical; the algebra has no connection with causal inference.

4 Linear layers over $\mathbb{R}_\varnothing$

Let a neuron have weights $W_{i1}, \ldots, W_{in} \in \mathbb{R}$ and bias b_i, and let $\tilde{x}$ be the input with $\varnothing$ replaced by 0.

Proposition 13 (Equivalence with zero imputation)**.** *If at least one $x_j \neq \varnothing$, then $\left(\bigoplus_j W_{ij} \odot x_j\right) \oplus b_i = \sum_j W_{ij}\tilde{x}_j + b_i$. If all $x_j = \varnothing$, the sum is $\varnothing$ and the layer outputs $\varnothing$ (the bias is not added).*

So, without rescaling, the layer *is* zero imputation; what it adds is the all-missing flag and, in the implementation, the per-neuron *coverage* k/n (the fraction of live synapses). Since inputs are usually standardised, zero imputation is mean imputation.

Proposition 14 (Rescaling)**.** *Let $S = \sum_j W_{ij}x_j$ and let L be a uniformly random subset of $\{1, \ldots, n\}$ of size $k \geq 1$ (inputs missing completely at random), with $S_L = \sum_{j\in L} W_{ij}x_j$.*

1. *$\mathbb{E}_L\left[\frac{n}{k}S_L\right] = S$: the factor n/k gives an unbiased estimate of the full sum.*
2. *If the x_j are independent with mean 0 and variance 1, then $\mathbb{E}\left[\frac{n}{k}S_L^2\right] = \mathbb{E}[S^2]$: the factor $\sqrt{n/k}$ preserves the second moment, whereas the factor n/k inflates it by n/k and no rescaling shrinks it by k/n.*

Proof. Each index lies in L with probability k/n, so $\mathbb{E}_L[S_L] = \frac{k}{n}S$ and, under the assumptions of (2), $\mathbb{E}[S_L^2] = \mathbb{E}_L \sum_{j\in L} W_{ij}^2 = \frac{k}{n}\sum_j W_{ij}^2 = \frac{k}{n}\mathbb{E}[S^2]$. □

The implementation offers both (`normalize="mean"` and `"variance"`). Neither is correct for correlated inputs in general, and neither replaces a model of the missing values.

5 Frozen parameters

$W \oplus \varnothing = W$ expresses "do not update this entry". Giving the update of a set of entries the value $\varnothing$ is a masked update, $W \leftarrow W + \Delta W$ on the complement of the mask, and produces the same numbers bit for bit.

For comparison, AdamW with a gradient that is merely zero applies $W_{t+1} = (1 - \eta\lambda)W_t - \eta\,\hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$, so once the moment estimate has decayed $W_t \approx (1 - \eta\lambda)^t W_0$ and the mean squared drift is $\left(1 - (1 - \eta\lambda)^t\right)^2 \mathbb{E}[W_0^2]$. For $\eta = 10^{-3}$, $\lambda = 2 \cdot 10^{-2}$, $t = 500$ this is about 10^{-4}. The drift is real but small, and excluding the entries from the update removes it without any algebra. Freezing parameters does not by itself address catastrophic forgetting, which concerns which parameters to protect and how the remaining capacity is used.

6 The tagged multiply–accumulate cell

`hardware/rnull_mac.v` implements acc $\leftarrow$ acc $\oplus$ $(w \odot x)$ on signed W-bit operands, with one tag bit per operand. On a null operand the multiplier inputs are forced to zero and the accumulator is not enabled. The output tag `acc_null` distinguishes "nothing was accumulated" from a real sum equal to 0; this is the only functional difference from a conventional sparse MAC with zero skipping. The accumulator has $2W + G$ bits; since $|wx| \le 2^{2W-2}$, any $N \le 2^{G+1} - 1$ products are summed exactly, and a sticky flag records overflow beyond that.

If weights and inputs are null independently with probabilities p_w, p_x, the fraction of skipped cycles is $1 - (1 - p_w)(1 - p_x)$. This is an *activity* figure. No synthesis, timing, area or power results exist for this cell; the tag bits themselves cost storage and routing.

7 Measured results

All numbers below are produced by the scripts in `benchmarks/` (NumPy reference implementation, fixed seeds) and stored in `benchmarks/results/`.

Skipped MAC operations (10^6 synapses, 5 seeds).

$p_w = p_x$	active (mean ± sd)	skipped, measured	$1 - (1 - p)^2$
0.10	810 047 ± 34	19.00%	19.00%
0.50	250 204 ± 176	74.98%	75.00%
0.70	89 970 ± 322	91.00%	91.00%
0.90	10 011 ± 110	99.00%	99.00%

Drift of frozen weights (10^5 weights $\sim N(0, 1)$, 80% frozen, 50 warm-up steps, then 500 steps; mean squared difference from the consolidated values).

η	λ	AdamW, zero gradient	masked update	null gradient
10^{-3}	10^{-2}	$3.0 \cdot 10^{-5}$	0	0
10^{-3}	$2 \cdot 10^{-2}$	$1.0 \cdot 10^{-4}$	0	0
10^{-2}	10^{-1}	$1.4 \cdot 10^{-1}$	0	0

The masked update and the null-gradient update agree on every weight (maximum absolute difference 0).

Output scale of a $64 \to 32$ layer (independent standard normal inputs; standard deviation relative to complete inputs), in agreement with Proposition 14.

missing	no rescaling	`"mean"`	`"variance"`
0%	1.00	1.00	1.00
30%	0.84	1.20	1.00
60%	0.63	1.60	1.00
90%	0.32	3.47	1.00

Prediction with incomplete inputs (one-hidden-layer MLP, 32 units; features removed completely at random at the same rate in training and test; mean ± sd over 5 seeds). "impute" is zero imputation of standardised inputs, which by Proposition 13 is the layer without rescaling; "impute+mask" appends the missingness indicators.

dataset (metric)	missing	impute	impute+mask	null, `"mean"`	null, `"variance"`
synthetic (R^2)	0%	0.878 ± 0.038	0.875 ± 0.038	0.878 ± 0.038	0.878 ± 0.038
	20%	0.763 ± 0.052	0.753 ± 0.044	0.753 ± 0.057	0.758 ± 0.059
	40%	0.694 ± 0.054	0.676 ± 0.058	0.671 ± 0.061	0.684 ± 0.052
	60%	0.566 ± 0.073	0.556 ± 0.054	0.518 ± 0.078	0.553 ± 0.083
diabetes (R^2)	0%	0.389 ± 0.085	0.404 ± 0.074	0.389 ± 0.085	0.389 ± 0.085
	20%	0.356 ± 0.105	0.378 ± 0.049	0.350 ± 0.092	0.340 ± 0.118
	40%	0.295 ± 0.100	0.283 ± 0.069	0.227 ± 0.073	0.293 ± 0.072
	60%	0.224 ± 0.117	0.166 ± 0.112	0.021 ± 0.108	0.173 ± 0.095
breast cancer (accuracy)	0%	0.957 ± 0.006	0.969 ± 0.020	0.957 ± 0.006	0.957 ± 0.006
	20%	0.936 ± 0.029	0.944 ± 0.027	0.946 ± 0.011	0.937 ± 0.030
	40%	0.934 ± 0.019	0.944 ± 0.020	0.934 ± 0.023	0.934 ± 0.019
	60%	0.917 ± 0.010	0.927 ± 0.032	0.911 ± 0.016	0.911 ± 0.020

Reading. Differences between methods are within one standard deviation almost everywhere. Null-aware rescaling does not improve on plain imputation on these problems, and the `"mean"` rescaling is clearly worse at high missing rates on the small regression set, consistent with the variance inflation of Proposition 14. These are small tabular problems with missingness completely at random; they do not cover structured or informative missingness, sequences, or large models.

8 Scope

What the construction provides: a consistent way to carry "absent" through sums, products and linear layers; an explicit all-missing output instead of a bias-only output; exact freezing of selected entries; and a hardware cell whose result tag distinguishes an empty sum from a zero sum.

What it does not provide, and earlier versions of the project material overstated: out-of-distribution detection (only the case of entirely missing input is flagged); a remedy for catastrophic forgetting; measured energy savings; or a predictive advantage over imputation.